



Непрерывная разработка и интеграция CI/CD

...

Лекция 1

Волков Михаил Юрьевич

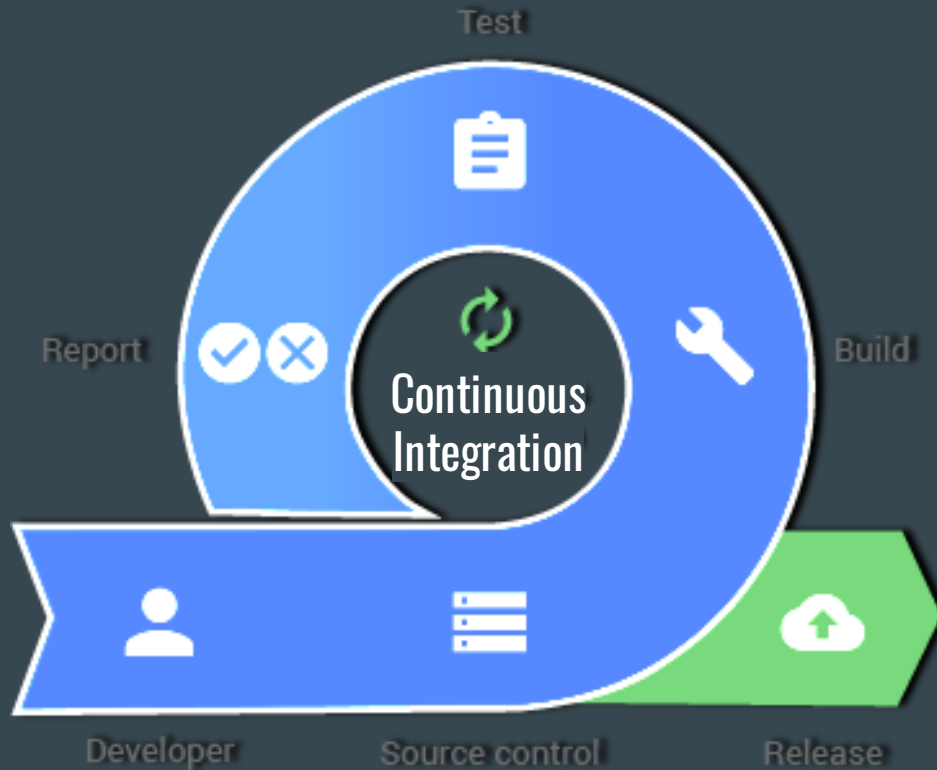
Тема 1

Понятие CI/CD особенности применения

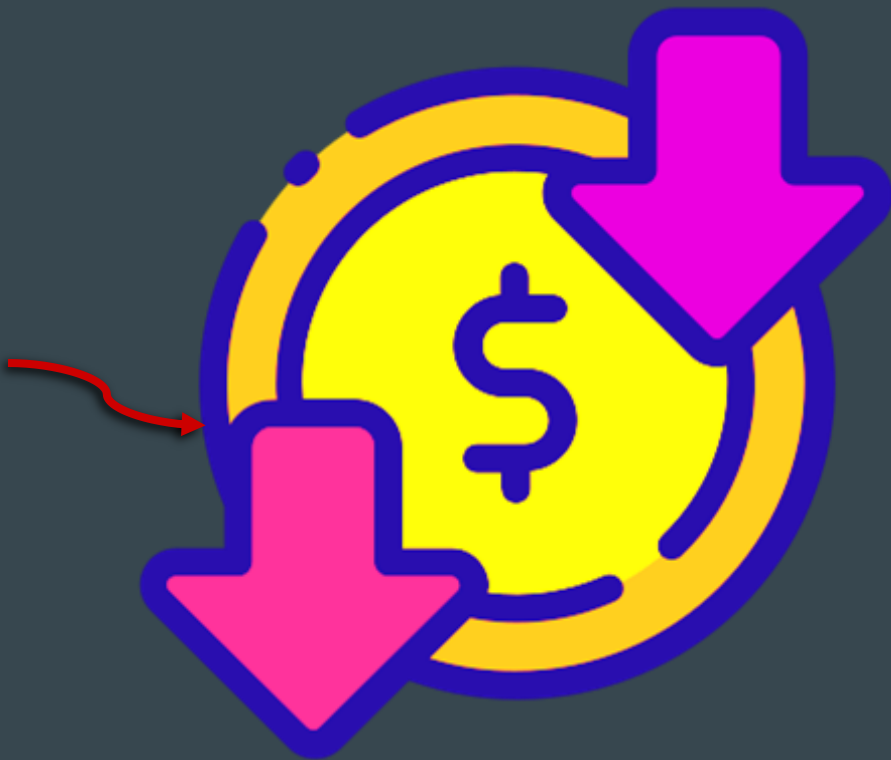
Данный курс предназначен для подготовки молодых специалистов в методологии CI/CD



Continuous Integration CI и User Acceptance Testing UAT



Критические ошибки - потеря финансов



Цели и затраты при разработке ПО

Целью является переход от ручного процесса предоставления программного обеспечения к:

1. Надежному
2. Предсказуемому
3. Контролируемому
4. Максимально автоматизированному процессу с четко понятными и количественно измеряемыми рисками

Большая часть затрат при создании успешного ПО приходится на время после выпуска первой версии. Сюда входит стоимость сопровождения, добавления новых средств и устранения ошибок.

Проблема развертывания программного обеспечения

Особое внимание на
процессах

Сборка

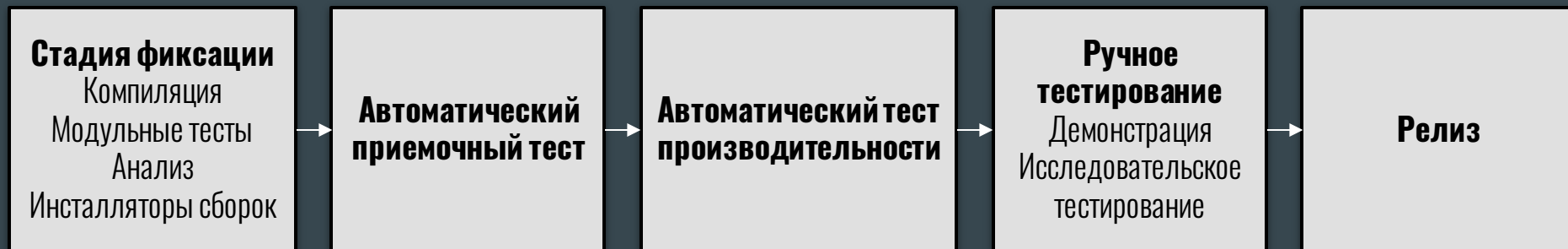
Установка

Тестирование

Развертывание



Конвейер развертывания как шаблон развертывания



Распространенные антишаблоны поставки релизов

Поставка релиза - это процесс, требующий интенсивного ручного труда.

Среда приложения часто создается индивидуально для каждого релиза администраторами или командой техподдержки.

1. Устанавливаются приложения сторонних производителей, от которых зависит релиз.
2. Релиз копируется в рабочую среду.
3. Конфигурационная информация копируется или создается администраторами веб-серверов, серверов приложений и сторонних компонентов.
4. Копируются ссылочные данные.
5. Приложение запускается в отладочной среде (по частям, если приложение распределенное или ориентированное на службы).

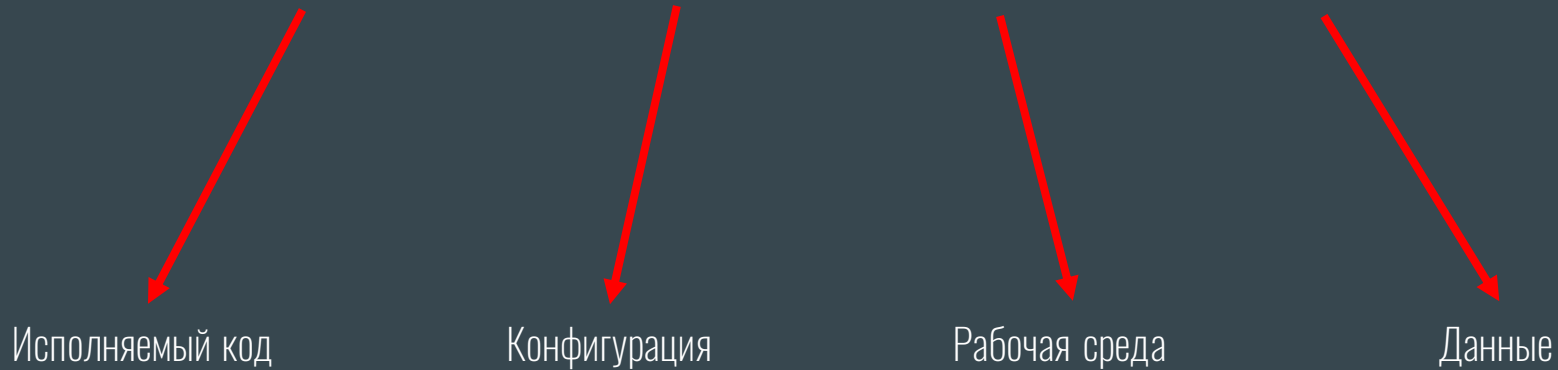
Антишаблон: развертывание ПО вручную

Основные признаки антишаблона:

1. Продуцирование большого объема документации, в которой подробно описывается, что нужно сделать и что может получиться неправильно.
2. Решение о работоспособности релиза принимается исключительно на основе результатов ручного тестирования.
3. Частые обращения к команде разработчиков для получения объяснений, почему в день поставки релиза приложение работает не так, как предполагается.
4. Частые изменения процедуры поставки релиза.
5. Конфигурации разных сред развертывания (тестовой, отладочной и рабочей) существенно отличаются друг от друга. Например, серверы приложений работают с разными пулами соединений и структурами файловых систем.
6. Пробные запуски релизов занимают много времени.
7. Результаты установки релизов непредсказуемые. Часто нужно возвращаться назад
8. и разбираться с возникшими проблемами.
9. Задержка персонала после рабочего дня для решения проблем, возникающих непосредственно в ходе поставки релиза.

Любое изменение должно инициировать обратную связь

Работающее приложение иногда полезно условно разделить на четыре компонента:



Обратная связь предполагает тестирование каждого изменения в полностью автоматическом режиме.

Любое изменение должно инициировать обратную связь

Тесты могут изменяться в зависимости от системы, однако они должны содержать как минимум следующие этапы:

1. Проверка процесса создания исполняемого кода. При его проверке разработчик убеждается в правильности синтаксиса исходного кода.
2. Приложение должно пройти модульное тестирование. Этим обеспечивается ожидаемое поведение кода приложения.
3. Тесты приложения должны удовлетворять определенным критериям качества, например требованиям покрытия кода и другим технологическим метрикам.
4. Приложение должно пройти приемочный тест функциональности. Этим обеспечивается удовлетворение деловых требований, ожидаемых от приложения.
5. Приложение должно пройти нефункциональные тесты. Это означает, что тест должен проверить производительность, доступность, безопасность и другие параметры приложения, не связанные напрямую с деловой логикой.
6. Приложение должно быть подвергнуто исследовательскому тестированию и продемонстрировано заказчику и группе пользователей. Обычно это ручная работа. На данной стадии процесса владелец продукта должен решить, не пропущены ли какие-либо средства. Иногда здесь же обнаруживаются ошибки в деловой логике.

Обратная связь должна срабатывать как можно быстрее



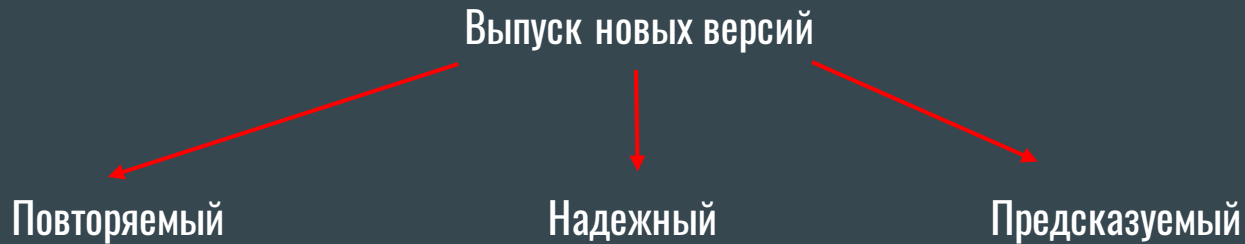
Для достижения быстрой обратной связи требуется использование автоматизации. В полностью автоматизированном процессе единственное ограничение - количество и параметры оборудования, которое можно задействовать для решения задачи.

Тесты на стадии фиксации конвейера

Тесты характеризуются следующими особенностями:

1. Выполняются быстро.
2. Покрывают около 75% кодовой базы.
3. Если любой тест терпит крах, значит, приложение содержит критическую ошибку и не должно быть пропущено на стадию выпуска.
4. Тесты по возможности должны быть независимыми по отношению к среде.

Преимущества непрерывного развертывания



1. Сокращение продолжительности цикла предоставления
2. Удешевление поддержки продукта
3. Уменьшение количества ошибок



Гибкость развертывания



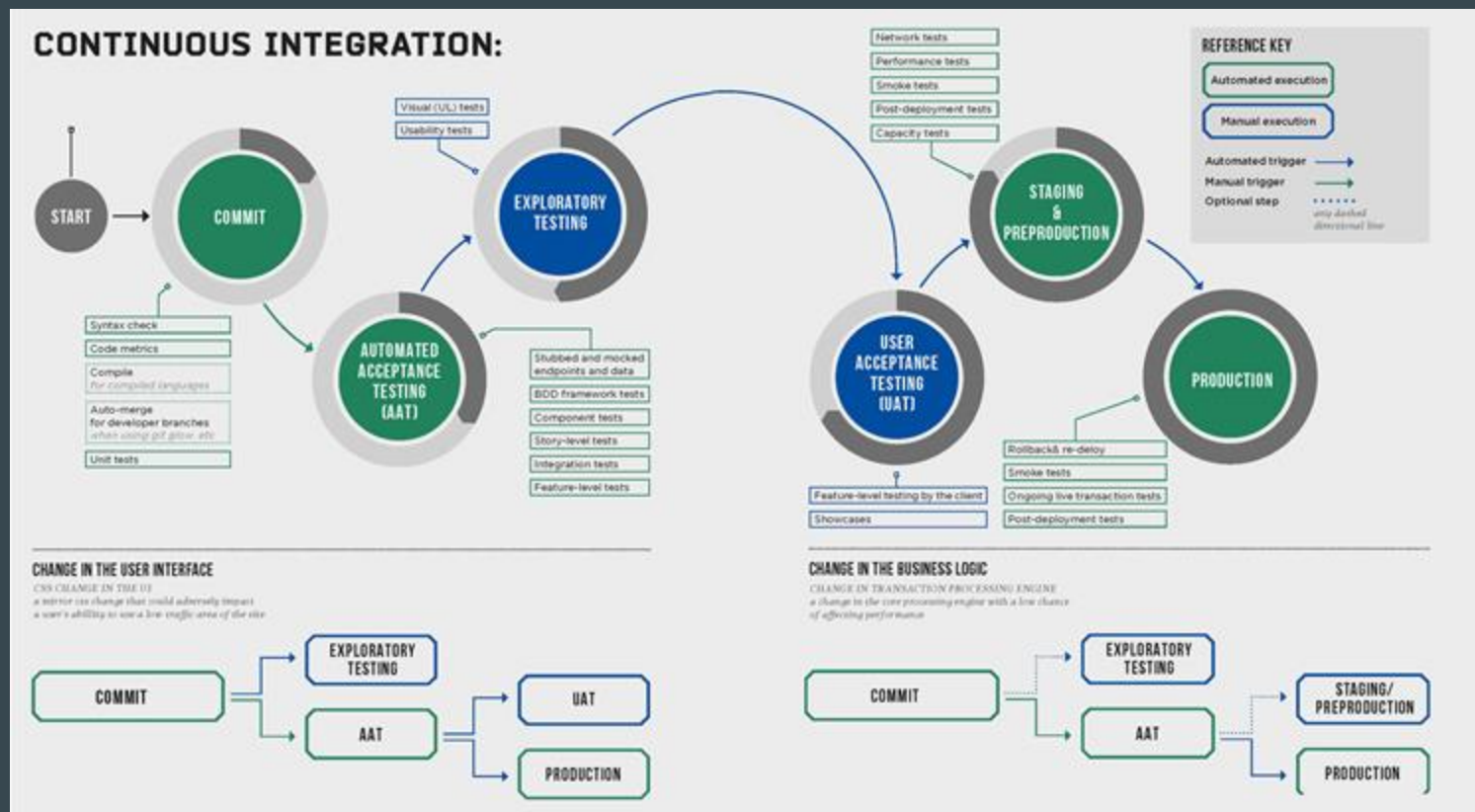
Установка приложения в новой среде должна быть простой задачей. В идеале она должна состоять лишь из определения конфигурационной информации, описывающей уникальные свойства среды. Затем запускается автоматизированный процесс, который готовит новую среду к развертыванию и устанавливает в нее выбранную версию приложения.

Релиз кандидат



Любое изменение может войти в релиз только после проверки его пригодности. Результирующий продукт может стать релизом, если он свободен от дефектов и удовлетворяет приемочным критериям, установленным заказчиком.

Каждое изменение может привести к выпуску новой версии



Технология непрерывной интеграции основана на идее как можно более частого выполнения процесса интегрирования. При соблюдении принципов непрерывной интеграции, приложение должно находиться в работоспособном состоянии.

Создайте надежный повторяющийся процесс развертывания

Развертывание должно быть легким и быстрым процессом.

Технология непрерывного развертывания базируется на трех составляющих:

1. Управление средой, в которой будет выполняться приложение (конфигурация оборудования, программное обеспечение, инфраструктура и внешние службы)
2. Установка в заданную среду правильной версии приложения
3. Конфигурирование приложения, включая данные и состояния



Создайте надежный повторяющийся процесс развертывания



Автоматизируйте все, что только можно

Автоматизировать некоторые процессы невозможно. В общем случае процесс сборки должен быть автоматизирован вплоть до точки, в которой необходимо принятие решения человеком.



Следующий шаг устранение дефекта



Сигнал тревоги бесполезен, если все проигнорируют его. Команда поставки должна быть дисциплинированной и устранять каждый дефект, как только он обнаружен.

Главные принципы движения DevOps

Цели движения DevOps те же, что и концепции непрерывного развертывания: поощрение тесного сотрудничества всех людей, имеющих отношение к приложению, для быстрого создания качественного и надежного программного продукта.



Базовая система непрерывной интеграции

Hudson - Microsoft Internet Explorer

Address: http://kohsuke.sfbay/hudson/

Hudson

ENABLE AUTO REFRESH

[New Job](#)

[Configure](#)

[Reload Config.](#)

Build Queue

Job	Status
hudson	Idle
jaxb-ri	Idle

Build Executor Status

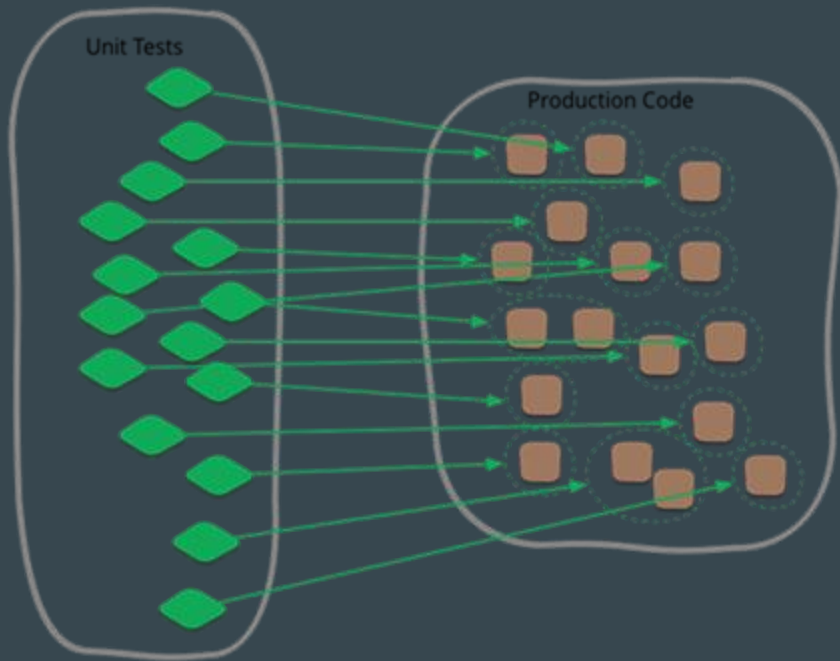
No.	Status
1	Idle
2	Idle
3	Building javanet-maven-repository-daemon #326
4	Building jaxb-ri #3181
5	Building glassfish #100
6	Idle

Job	Last Success	Last Failure	Last Duration
Common annotations	4 days (#16)	9 months (#3)	39 seconds
hsh	6 months (#11)	10 months (#2)	59 seconds
dtd-parser	6 months (#8)	N/A	1 minute
f	28 days (#506)	1 month (#567)	7 minutes
f.weekly	6 days (#53)	13 days (#52)	5 minutes
glassfish	4 hours (#104)	1 day (#88)	1 hour
hudson	4 minutes (#201)	N/A	1 minute
stack-commons	12 days (#19)	16 days (#5)	14 seconds
ispec	3 days (#55)	9 hours (#54)	1 minute
java-ee-xmi community discussion updater	4 minutes (#16146)	10 hours (#16125)	1 minute
java.net.acl.processor	18 hours (#162)	N/A	0 seconds



Создайте полный набор автоматических тестов

Существуют три типа тестов, интересных с точки зрения непрерывной интеграции: модульные тесты, тесты компонентов и приемочные тесты.



Создайте полный набор автоматических тестов

Unit.net



Использование программ непрерывной интеграции

На рынке существует огромное количество программных продуктов, предоставляющих инфраструктуру автоматизации процессов сборки и тестирования. Наиболее фундаментальная функция программ непрерывной интеграции опрос системы управления версиями для обнаружения выполненных фиксаций и, если таковые есть, подтверждение последней версии приложения, запуск сценариев сборки и тестирования и оповещение исполнителя о результатах.

Базовые возможности

Программное обеспечение сервера непрерывной интеграции состоит из двух компонентов. Первый компонент долгоживущий процесс, выполняющий простые операции через регулярные промежутки времени. Второй компонент выдает сводку результатов выполненных процессов, оповещает об успешности или неуспешности сценариев сборки и тестирования и предоставляет доступ к отчетам тестов, инсталляторам.

Базовые возможности

Большинство серверов непрерывной интеграции содержат веб-сервер, предоставляющий список выполненных сборок и позволяющий просматривать отчеты о результатах каждой сборки.

The screenshot shows the Hudson web interface in a Microsoft Internet Explorer browser window. The address bar shows the URL <http://kohauke.sfbay/hudson/>. The page title is "Hudson".

On the left sidebar, there are links for "New Job", "Configure", and "Reload Config". Below these, there is a "Build Queue" section showing a list of jobs with status icons. Further down is the "Build Executor Status" section, which shows a table of build executors and their current status.

The main content area displays a table of build jobs. The table has columns for "Job", "Last Success", "Last Failure", and "Last Duration". The jobs listed include "Common annotations", "bah", "old-parser", "fi", "fi (weekly)", "glassfish", "hudson", "stack-commons", "jacex", "java-ws-xml-community", and "java.net.ecl.processor". Each job entry includes a status icon (blue for success, red for failure) and a link to view the job details.

Job	Last Success	Last Failure	Last Duration
Common annotations	4 days (#16)	9 months (#2)	39 seconds
bah	6 months (#11)	10 months (#2)	59 seconds
old-parser	6 months (#3)	N/A	1 minute
fi	28 days (#50)	1 month (#567)	7 minutes
fi (weekly)	6 days (#33)	13 days (#32)	5 minutes
glassfish	4 hours (#106)	1 day (#88)	1 hour
hudson	4 minutes (#201)	N/A	1 minute
stack-commons	12 days (#19)	16 days (#5)	14 seconds
jacex	3 days (#55)	9 hours (#64)	1 minute
java-ws-xml-community	4 minutes (#16149)	10 hours (#16125)	1 minute
java.net.ecl.processor	18 hours (#162)	N/A	0 seconds

Не регистрируйте изменения в нерабочей сборке



Всегда выполняйте все тесты фиксации локально перед самой фиксацией либо заставьте делать это сервер непрерывной интеграции



TeamCity Cloud

New Platform.
Same great CI/CD.

JET
BRAINS

electric-cloud/
electriccommander



ElectricCommander automates and accelerates the application development (build-test-deploy) process and its supporting IT resources (physical, virtual, or public/private cloud).

17

Contributors

2

Issues

32

Stars

26

Forks



Экстремальное программирование

Непрерывная интеграция — это одна из двенадцати базовых методик экстремального программирования, и в таком качестве она дополняет другие методики этой технологии и сама дополняется ими. Непрерывная интеграция оказывает огромное влияние на работу команды, даже если в проекте не применяются другие методики экстремального программирования.



Отмена сборки из-за нарушения архитектурных ограничений

Часто архитектура системы обладает некоторыми особенностями, о которых разработчики быстро забывают. Одно из решений этой проблемы состоит в создании тестов этапа фиксации, проверяющих, не нарушены ли требования архитектуры. Следует признать, что данный прием чисто тактический.

Отмена сборки из-за медлительности тестов



Если тесты фиксации выполняются долго, они могут существенно ухудшить производительность команды, вынуждая ожидать завершения процессов сборки и тестирования. Это, в свою очередь, способствует откладыванию регистраций. Команда начинает накапливать изменения, и каждая регистрация становится все более сложной. При этом возрастает вероятность конфликтов слияния, появления ошибок и краха тестов. Все это еще больше замедляет разработку.

Отмена сборки из-за предупреждений и нарушений стиля кодирования

Инструменты, предназначенных для проверки кода:

1. Simian инструмент, обнаруживающий дублирование кода в большинстве популярных языков программирования (включая простой текст).
2. JDepend для Java и его коммерческий аналог NDepend для .NET генерируют множество полезных (иногда не очень) метрик качества кода.
3. CheckStyle обнаруживает плохие стили кодирования, например открытые конструкторы в классах утилит, вложенные блоки и длинные строки. Этот же инструмент выявляет распространенные источники ошибок и бреши в системе безопасности. Его легко расширить. FxCop является аналогом для .NET.
4. FindBugs альтернатива инструмента CheckStyle для платформы Java, содержащая аналогичный набор проверок.



Распределенные команды



Спасибо за внимание !

